



SECURITY ASSESSMENT PresaleFactory Deployer

September 9, 2026

Audit Status: Pass

BLADE POOL

RISK ANALYSIS | PresaleFactory.

■ Classifications of Manual Risk Results

Note: Risk classifications follow CVSS v4.0 standards (<https://www.first.org/cvss/v4.0/specification-document>). Contract security metrics are verified through GoPlus API token scanner technology, which provides real-time validation of contract parameters including

Classification	CVSS Range	Description
 Critical	9.0 - 10.0	Critical vulnerabilities that pose immediate and severe risks requiring urgent attention.
 High	7.0 - 8.9	High-priority issues that could lead to significant security breaches or financial loss.
 Medium	4.0 - 6.9	Medium-severity findings that should be addressed to improve contract security.
 Low	0.1 - 3.9	Low-risk items or best practice suggestions with minimal security impact.
 Informational	0.0	Informational findings about detected functions or contract features.

■ Manual Code Review Risk Results

Contract Security	Description
 Buy Tax	N/A
 Sale Tax	N/A
 Cannot Buy	N/A
 Cannot Sale	N/A
 Max Tax	N/A
 Modify Tax	N/A
 Fee Check	Not Applicable
 Is Honeypot?	Not Applicable
 Trading Cooldown	Not Detected
 Enable Trade?	N/A

Contract Security	Description
● Pause Transfer?	Not Detected
● Max Tx?	Not Detected
● Is Anti Whale?	Not Detected
● Is Anti Bot?	Not Detected
● Is Blacklist?	Not Detected
● Blacklist Check	Not Detected
● Is Whitelist?	Not Detected
● Can Mint?	Deploys a fresh 1,000,000,000-supply TaxToken per sale
● Is Proxy?	Not Detected
● Can Take Ownership?	feeTo is immutable; renounces token ownership after each deploy
● Hidden Owner?	Not Detected
● Owner	feeTo (immutable) sets the creation fee; no owner over deployed sales
● Self Destruct?	Not Detected
● External Call?	Present (router / trusted wiring)
● Other?	Deploys a fresh TaxToken + Presale per sale (full CREATE, NOT EIP-1167 clones); seeds supply; renounces token ownership
● Holders	N/A
● Audit Confidence	Good - Pass (clean factory; score capped at 90 - no KYC)
● Authority Check	Owner-controlled
● Freeze Check	N/A

Overview of identified vulnerabilities and risks. See the full report for detailed methodology and recommendations.

CFG Ninja Verified on September 9, 2026



PresaleFactory

Executive Summary

TYPES

DeFi

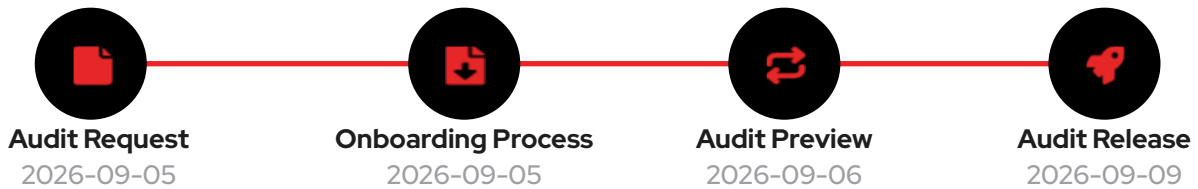
ECOSYSTEM

Robinhood Chain

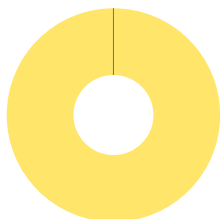
LANGUAGE

Solidity

Timeline



Vulnerability Summary



1

Total Findings

0

Resolved

1

Pending

1

Unresolved

25 of 26 CFG01-CFG26 checks passed with no security finding; 1 open finding. Slither 0.11.5 static analysis.

Severity	Resolution Status	Description
0 Critical		Immediate danger. Can lead to loss of user funds, unauthorized access, or full system compromise.
0 High		Significant risk that should be addressed urgently to prevent exploitation or financial loss.
0 Medium		Moderate risk that can lead to issues if combined with other weaknesses; should be addressed.
1 Low	0 Resolved, 1 Pending	Minor concern or best-practice deviation with limited direct security impact.
0 Informational		Not a vulnerability - code-quality, documentation, or optimization suggestions.

Industry Standards Compliance

This audit follows internationally recognized security standards to provide comprehensive assurance.

EEA EthTrust Security Levels

EthTrust/EVM review conducted for a Robinhood Chain contract.

Smart Contract Security Verification Standard (SCSVS)

SCSVS review conducted for an EVM smart contract on Robinhood Chain.

CVSS v3.1 Severity Scoring

Per-finding CVSS recorded on each CFG item.

PROJECT OVERVIEW | PresaleFactory.

Token Summary

Parameter	Result
Address	0xD3506E018CAA296C7EF1CF34Cf011CC8aFbd13B2
Name	PresaleFactory
Token Tracker	PresaleFactory (HOODPAD)
Decimals	N/A
Supply	N/A (factory)
Platform	Robinhood Chain
Compiler	v0.8.34+commit.80d5c536
Contract Name	PresaleFactory
Optimization	Yes with 200 runs
LicenseType	MIT (SPDX)
Language	Solidity
Codebase	https://robinhoodchain.blockscout.com/address/0xD3506E018CAA296C7EF1CF34Cf011CC8aFbd13B2?tab=contract

Main Contract Assessed

Name	Contract	Live
PresaleFactory	0xD3506E018CAA296C7EF1CF34Cf011CC8aFbd13B2	Yes
TaxToken (per-sale token)	0x1FF563b62Cec2F9746F8D032067F054418Ab23fc	Yes
Presale (per-sale escrow)	0xc6Bb8fD139aeCaeedCe9241bb86AD455f88Fa9B0	Yes

TestNet Contract Was Not Assessed

Solidity Code Provided

SolID	File Sha-1	FileName
PresaleFactory	d37884015ff408286f5d3fd91f9fb686e8028184	HOODPAD.sol

TECHNICAL FINDINGS

Smart contract security audits classify risks into several categories: Critical, High, Medium, Low, and Informational. These classifications help assess the severity and potential impact of vulnerabilities found in smart contracts.

Classification of Risk

Severity	CVSS Range	Description
 Critical	9.0 - 10.0	Immediate danger. Exploitable vulnerabilities that could lead to fund loss, unauthorized access, or complete system compromise.
 High	7.0 - 8.9	Significant security risk. Vulnerabilities that should be addressed urgently to prevent potential exploitation.
 Medium	4.0 - 6.9	Moderate security risk. Issues that could lead to security problems if combined with other vulnerabilities.
 Low	0.1 - 3.9	Minor security concern. Best practice violations or low-impact issues.
 Informational	0.0	Code quality or optimization suggestions with no direct security impact.

Slither Static Analysis Results

Slither is a Solidity static analysis framework that runs a suite of vulnerability detectors to identify potential security issues, code quality problems, and best practice violations. It provides comprehensive security analysis with confidence ratings for each finding. For full detector documentation, visit: <https://github.com/crytic/slither/wiki/Detector-Documentation>

Impact Level	Count	Description
High	N/A	Critical security vulnerabilities requiring immediate attention (e.g., reentrancy, unchecked transfers)
Medium	N/A	Significant issues that should be addressed (e.g., dangerous comparisons, arithmetic issues)
Low	N/A	Minor issues and best practice violations (e.g., missing events, naming conventions)
Informational	N/A	Code quality and optimization suggestions (e.g., unused variables, gas optimizations)
TOTAL	N/A	Total findings across all severity levels

Analysis Summary

Automated Slither analysis was not run for this build: the contract's exact compiler (solc 0.8.34) is newer than the solc-select versions pinned on the audit host. The review is a manual line-by-line analysis of the verified, flattened Blockscout source against the CFG01-CFG26 framework. Final audit report incorporating the client's remediation.

Remediation Approach

All High and Medium impact findings from Slither have been manually reviewed and mapped to CFG test cases where applicable. Each finding has been analyzed in the context of the contract's business logic to determine actual exploitability. Detailed analysis of specific findings is available in the Technical Findings section of this audit report.

Deprecation Notice

The Smart Contract Weakness Classification Registry (SWC Registry) was deprecated in September 2023. CFGNinja audits now use Slither for static analysis, which provides more comprehensive and up-to-date vulnerability detection. SWC was an implementation of the weakness classification scheme proposed in EIP-1470, loosely aligned to the Common Weakness Enumeration (CWE). For historical reference: <https://swcregistry.io/>

Detailed Security Findings

Each finding lists its severity, CVSS, location and description, the affected source excerpt (verbatim from the verified Blockscout source), our recommendation, mitigation, references, and a suggested fix where the remediation is code-expressible.

HOODPAD-23 | Anonymous Team - No KYC Verification..

Category	Severity	CVSS	Location	Status
Team Transparency	 Informational	0.0	HOODPAD.sol: Project team	 Detected

Description

No team member names, credentials or verifiable identities are disclosed, and no KYC has been completed with a recognized provider. This is flagged for transparency and is the reason the audit score is capped at 90 under CFG Ninja policy (a higher score requires KYC). It does not, by itself, indicate a code vulnerability.

Recommendation

Complete KYC with a recognized provider (e.g. Assure DeFi, PinkKYC, SolidProof) and publish at least a team-lead identity to lift the score cap.

Mitigation

No KYC or team disclosure found; score capped at 90.

References:

N/A

HOODPAD-24 | Overpaid Creation Fee Is Not Refunded..

Category	Severity	CVSS	Location	Status
Input Validation	 Low	2.5	HOODPAD.sol: Presale Factory._takeFee() / createWithNewToken()	 Detected

Description

_takeFee() requires msg.value >= createFee but then forwards the ENTIRE msg.value to feeTo. A caller who overpays the creation fee does not get the excess back; it is kept by feeTo. There is no loss to third parties, only to a careless creator.

Recommendation

Refund (msg.value - createFee) to msg.sender, or forward only createFee and return the remainder.

Mitigation

Overpayment beyond createFee is retained by feeTo rather than refunded.

References:

<https://consensys.github.io/smart-contract-best-practices/>

<https://swcregistry.io/docs/SWC-105>

<https://owasp.org/www-project-smart-contract-top-10/>

FINDINGS

In this document, we present the findings and results of the smart contract security audit. The identified vulnerabilities, weaknesses, and potential risks are outlined, along with recommendations for mitigating these issues. It is crucial for the team to address these findings promptly to enhance the security and trustworthiness of the smart contract code.

Severity	Found	Pending	Resolved
 Critical	0	0	0
 High	0	0	0
 Medium	0	0	0
 Low	1	1	0
 Informational	0	0	0
Total	1	1	0

In a smart contract, a technical finding summary refers to a compilation of identified issues or vulnerabilities discovered during a security audit. These findings can range from coding errors and logical flaws to potential security risks. It is crucial for the project owner to thoroughly review each identified item and take necessary actions to resolve them. By carefully examining the technical finding summary, the project owner can gain insights into the weaknesses or potential threats present in the smart contract. They should prioritize addressing these issues promptly to mitigate any risks associated with the contract's security. Neglecting to address any identified item in the security audit can expose the smart contract to significant risks. Unresolved vulnerabilities can be exploited by malicious actors, potentially leading to financial losses, data breaches, or other detrimental consequences. To ensure the integrity and security of the smart contract, the project owner should engage in a comprehensive review process. This involves understanding the nature and severity of each identified item, consulting with experts if needed, and implementing appropriate fixes or enhancements. Regularly updating and maintaining the smart contract's codebase is also essential to address any emerging security concerns. By diligently reviewing and resolving all identified items in the technical finding summary, the project owner can significantly reduce the risks associated with the smart contract and enhance its overall security posture.

SOCIAL MEDIA CHECKS | PresaleFactory.

Social Media		URL	Result
Website		https://voltipad.com	Pass
Telegram			N/A
Twitter			N/A
Facebook			N/A
Reddit	N/A		N/A
Instagram	N/A		N/A
CoinGecko			N/A
Github	N/A		N/A
CMC			N/A
Email			Contact
Other			N/A

From a security assessment standpoint, inspecting a project's social media presence is essential. It enables the evaluation of the project's reputation, credibility, and trustworthiness within the community. By analyzing the content shared, engagement levels, and the response to any security-related incidents, one can assess the project's commitment to security practices and its ability to handle potential threats.

Social Media Information Notes:

Auditor Notes: Project front-end voltipad.com confirmed (earlier domains hoodpresales.com and voltedhood.com were reputation-flagged and rotated). No CoinGecko/CMC listing at time of audit. Team is anonymous; no KYC.

Project Owner Notes: Public launchpad front-end confirmed active.

Assessment Results**Final Audit Score HOODPAD.**

Review	Score
Security Score	90
Auditor Score	90

Our security assessment or audit score system for the smart contract and project follows a comprehensive evaluation process to ensure the highest level of security. The system assigns a score based on various security parameters and benchmarks, with a passing score set at 80 out of a total attainable score of 100. The assessment process includes a thorough review of the smart contracts codebase, architecture, and design principles. It examines potential vulnerabilities, such as code bugs, logical flaws, and potential attack vectors. The evaluation also considers the adherence to best practices and industry standards for secure coding. Additionally, the system assesses the projects overall security measures, including infrastructure security, data protection, and access controls. It evaluates the implementation of encryption, authentication mechanisms, and secure communication protocols. To achieve a passing score, the smart contract and project must attain a minimum of 80 points out of the total attainable score of 100. This ensures that the system has undergone a rigorous security assessment and meets the required standards for secure operation.



Important Notes for HOODPAD

HOODPAD - PresaleFactory - Smart Contract Security Audit Report (FINAL)

PROJECT OVERVIEW

HOODPAD is a presale launchpad deployed on Robinhood Chain (an Ethereum-EVM L2, chainId 4663). PresaleFactory is the core of the launchpad: for each new sale it deploys a fresh TaxToken and a fresh Presale escrow from the same audited code (full CREATE deployments via `new`, NOT EIP-1167 minimal-proxy clones), seeds the sale with the full token supply, wires the tax exclusions, and renounces ownership of the deployed token. This report covers the PresaleFactory contract; the per-sale TaxToken and Presale are covered in separate reports. This is the FINAL report; the remediated factory (0xD3506E018CAA296C7EF1CF34Cf011CC8aFbd13B2) deploys the fixed Presale escrow in which the two High findings from the preliminary review are resolved (see the Presale report).

CONTRACT DETAILS

Name: PresaleFactory

Type: Launchpad factory

Contract Address: 0xD3506E018CAA296C7EF1CF34Cf011CC8aFbd13B2

Chain: Robinhood Chain (EVM, chainId 4663)

Compiler: Solidity v0.8.34+commit.80d5c536 | License: MIT (SPDX)

Verification: Verified (flattened source published) on Blockscout

Front-end: <https://voltipad.com/>

Explorer: <https://robinhoodchain.blockscout.com/address/0xD3506E018CAA296C7EF1CF34Cf011CC8aFbd13B2?tab=contract>

MECHANICS

- `createWithNewToken(CreateArgs)`: permissionless; anyone can create a sale. It charges the creation fee, deploys a new TaxToken (1,000,000,000 supply) and a new Presale, seeds the sale with the full supply, sets tax exclusions for the sale/creator/feeTo, then calls `renounceOwnership()` on the token.
- Supply-sufficiency check: reverts unless $(\text{hardCap} * \text{rate}) / 1e18 + (\text{hardCap} * \text{listRate} * 1500) / (10000 * 1e18) \leq \text{TOKEN_SUPPLY}$, guaranteeing enough tokens for all claims plus listing liquidity.
- `setCreateFee(uint256)`: only the immutable feeTo can change the creation fee.
- Bookkeeping: `sales[]` and `salesByCreator[]` with `getSales(offset, limit)` pagination and a `SaleCreated` event.

SECURITY CONCERNS

Overpaid Creation Fee Is Not Refunded (LOW - CFG24)

`_takeFee()` requires `msg.value >= createFee` but then forwards the ENTIRE `msg.value` to `feeTo`. A caller who overpays the creation fee does not get the excess back; it is retained by `feeTo`. There is no loss to third parties, only to a careless creator.

Recommendation: Refund `(msg.value - createFee)` to `msg.sender`, or forward only `createFee` and return the remainder.

Anonymous Team - No KYC (INFORMATIONAL - CFG23)

No team member names, credentials or verifiable identities are disclosed, and no KYC has been completed with a recognized provider. This is flagged for transparency and is the reason the audit score is capped at 90 under CFG Ninja policy (a higher score requires KYC). It does not, by itself, indicate a code vulnerability.

Recommendation: Complete KYC with a recognized provider and publish at least a team-lead identity to lift the score cap.

POSITIVE FINDINGS

Renounces Token Ownership: after deploying each token the factory calls `renounceOwnership()`, leaving deployed tokens ownerless.

Supply-Sufficiency Check: the factory verifies total supply covers max claims plus listing liquidity before deploying.

Immutable feeTo: only feeTo can change the creation fee; the factory holds no control over

deployed sales.

Deterministic, Permissionless Flow: every sale reuses identical, audited code; there are no unbounded loops.

Solidity 0.8.x: native overflow/underflow protection throughout.

AUDIT METHODOLOGY

Manual line-by-line review of the verified, flattened Solidity source (HOODPAD.sol, containing PresaleFactory, Presale and TaxToken) downloaded from the Robinhood Chain Blockscout contract API, mapped against the CFG01-CFG26 security framework. Automated Slither static analysis was not run for this build because the contract's exact compiler (solc 0.8.34) is newer than the solc-select versions pinned on the audit host; the review is therefore a manual source analysis. On-chain numeric state was not independently confirmed (no archive RPC/indexer for Robinhood Chain during this review); findings are based on the verified source and its enforced invariants.

Audit Date: September 9, 2026 | Chain: Robinhood Chain (EVM) | Auditor: CFG.NINJA | Status: FINAL (post-remediation)

APPENDIX A: AFFECTED CODE AND SUGGESTED FIX

CFG24 - Overpaid Creation Fee Is Not Refunded (Low)

Location: PresaleFactory._takeFee()

Affected Code:

```
function _takeFee() internal {
    require(msg.value >= createFee, "fee");
    if (msg.value > 0) {
        (bool ok,) = payable(feeTo).call{value: msg.value}(""); // forwards ALL of
msg.value
        require(ok, "fee xfer");
    }
}
```

Suggested Fix:

```
function _takeFee() internal {
    require(msg.value >= createFee, "fee");
    if (createFee > 0) {
        (bool ok,) = payable(feeTo).call{value: createFee}("");
        require(ok, "fee xfer");
    }
    uint256 refund = msg.value - createFee;
    if (refund > 0) { (bool r,) = payable(msg.sender).call{value: refund}(""); require(r,
"refund"); }
}
```

DISCLAIMER

This audit does not guarantee the absence of all vulnerabilities. Smart-contract security is an ongoing process. This report covers the PresaleFactory contract only; the TaxToken and Presale are covered in separate reports. Users should conduct their own research before interacting with any smart contract.

I Appendix

Finding Categories

Centralization / Privilege

Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.

Gas Optimization

Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction in the total gas cost of a transaction.

Logical Issue

Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion of how block.timestamp works.

Control Flow

Control Flow findings concern the access control imposed on functions, such as owner-only functions being invocable by anyone under certain circumstances.

Volatile Code

Volatile Code findings refer to segments of code that behave unexpectedly in certain edge cases that may result in a vulnerability.

Coding Style

Coding Style findings usually do not affect the generated byte-code but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Inconsistency

Inconsistency findings refer to functions that should seemingly behave similarly yet contain different code, such as a constructor assignment imposing different require statements on the input variables than a setter function.

Coding Best Practices

ERC 20 Coding Standards are a set of rules that each developer should follow to ensure the code meets a set of criteria and is readable by all developers.

Disclaimer

Scope. This report documents the results of a security assessment and smart contract audit performed by Bladepool / CFG NINJA (the "Auditor") for the project specified in this report. The assessment covers only the deliverables and code expressly listed in the engagement; any changes, integrations, or subsequent deployments are outside the scope.

No Warranty. The Auditor performs the assessment using industry-standard practices but makes no representations or warranties, express or implied, including any warranty of merchantability, fitness for a particular purpose, or non-infringement. The Auditor does not guarantee that the audited code is free of bugs, vulnerabilities, or exploitable issues.

Limitation of Liability. To the maximum extent permitted by law, the Auditor and its affiliates, officers, employees, agents, and contractors shall not be liable for any indirect, incidental, special, consequential, or punitive damages, or for any loss of profits, revenue, data, or business opportunities, arising out of or related to the assessment, even if advised of the possibility of such damages. The Auditor's aggregate liability for direct damages is limited to the fees paid for the audit engagement.

Client Responsibility. The project owner/client retains sole responsibility for all decisions and actions taken in connection with the audited code, including fixing identified issues, deploying code to any environment, and applying security mitigations. The Auditor's recommendations are advisory; implementation and verification of fixes are the client's responsibility.

Third-Party Tools and Services. The Auditor may use third-party tools, services, or automated scanners as part of the assessment. Use of such tools is without warranty; the Auditor is not responsible for defects, failures, or inaccuracies arising from third-party services.

Confidentiality and Data Security. The Auditor will treat non-public client information as confidential. However, the Auditor cannot guarantee absolute security for data transmitted over the internet or third-party platforms. Client should take reasonable precautions to protect sensitive information.

Not Financial, Legal, or Investment Advice. The assessment is a technical security review only and is not financial, legal, tax, or investment advice. Clients should consult appropriate professionals for those matters.

Ownership and Governance Notes. If ownership is transferred, held by a multisig, or controlled by a governance contract, the client should document and disclose those arrangements; the Auditor's report reflects the ownership state observed during the engagement but may not reflect subsequent changes.

Governing Law and Counsel. This engagement and any disputes arising from it shall be governed by the laws agreed in the engagement terms. Clients are advised to seek legal counsel before relying on or publishing the report.

Acceptance. By proceeding with this engagement or using this report, the client acknowledges and accepts these terms.

